NESNE YÖNELİMLİ PROGRAMLAMA · HAFTA 7

Uygulamalı Tekrar

1–6. haftalar tek projede: bir kütüphane sistemini adım adım kuruyoruz

Hafta 1–2 | Hafta 3–4 | Hafta 5 | Hafta 6

```
kutuphane.py
kutup += Kitap.kayittan(satir)
kutup.odunc_ver(ayse, isbn)
print(ayse)

# Ayşe · 1 kitap
```

Erhan AKAGÜNDÜZ

BU HAFTA

Dersin sonunda neler yapabileceksiniz?

Temeller

Doğru veri yapısını seçmek ve OOP'ye geçişi gerekçelendirmek

Sınıf ve nesne

__init__, self, sınıf değişkenleri ve üç metot türünü birlikte kullanmak

Kapsülleme

Kuralları property ve özel niteliklerle korumak

Magic metotlar

Sınıfları print, ==, len, in ve += ile doğal hâle getirmek

☆ **Renk kodu:** yeşil = hafta 1–2, mavi = hafta 3–4, mor = hafta 5, turuncu = hafta 6.

NEREDEN GELDİK?

Altı hafta, altı yapı taşı

HAFTA 1

Veri yapıları, kapsam

Projede: kitapları ISBN anahtarlı sözlükte tutmak

HAFTA 2

Paradigmalar

Projede: prosedürel başlangıçtan OOP'ye geçiş

HAFTA 3

__init__ ve self

Projede: Kitap, Uye, Kutuphane nesneleri

HAFTA 4

Değişkenler ve metotlar

Projede: toplam sayaç, LIMIT, statik ve sınıf metodu

HAFTA 5

Kapsülleme

Projede: korunan ISBN, doğrulanan adet, sızmayan liste

HAFTA 6

Magic metotlar

Projede: print, ==, len, in, [ ], for ve +=

Bugün hepsi tek projede birleşiyor. **Gelecek hafta:** genel tekrar ve ara sınav.

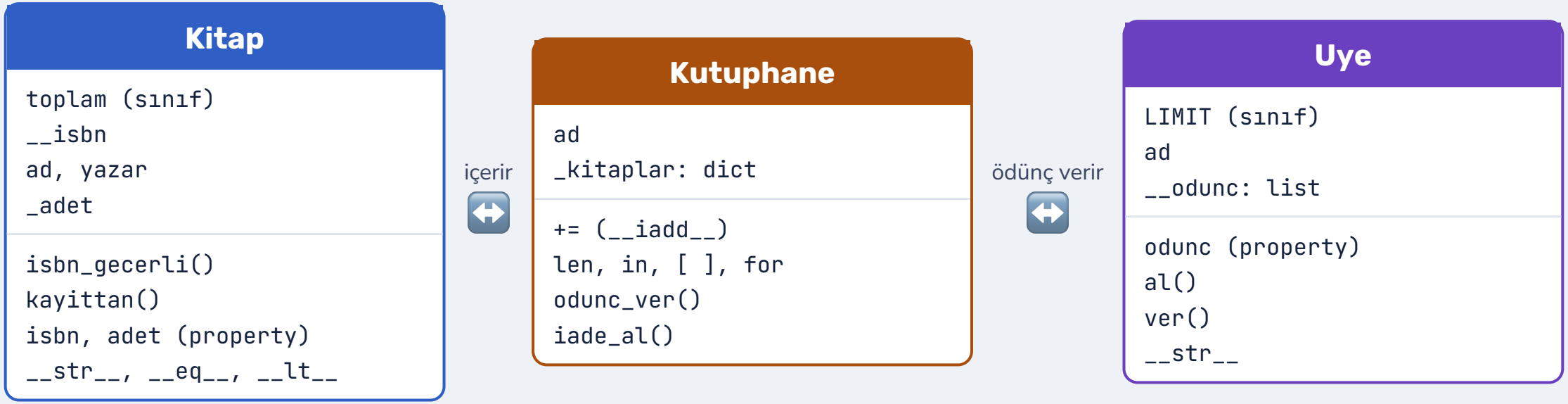
PROJE

Kütüphane sistemi: gereksinimler

- Kitaplar **ISBN** ile hızlıca bulunur — Hafta 1
- Kitap, üye, kütüphane ayrı **nesneler** — Hafta 3
- Kaç kitap nesnesi oluşturuldu **sayılır** — Hafta 4
- Bir üye en fazla **2 kitap** alabilir — Hafta 4
- ISBN **13 haneli** olmalı ve değişmez — Hafta 5
- Adet **negatif** olamaz — Hafta 5
- Aynı kitap tekrar eklenince **adet artar** — Hafta 6
- Kitaplar **ada göre sıralı** listelenir — Hafta 6

TASARIM

Üç sınıf, net görevler



Kural nerede yaşıyorsa orada kontrol edilir: **adet** kuralı Kitap'ta, **limit** kuralı Uye'de, **stok** kontrolü Kutuphane'de.

01

BÖLÜM 1

Temeller

Hafta 1–2: veri yapıları ve paradigmalar

list · tuple · set · dict · prosedürel · OOP'ye geçiş

Her ihtiyaca doğru yapı

```
katalog = {"9789750719387": "Kürk Mantolu Madonna"}
kategoriler = {"roman", "tarih", "roman"}
kayit = ("9789750719387", "Sabahattin Ali")
bekleme = ["Ayşe", "Can"]

print(katalog["9789750719387"])
print(sorted(kategoriler))
print(kayit[1])
bekleme.append("Ece"); print(bekleme)
kayit[1] = "Başkası"
```

▷ ÇIKTI

```
Kürk Mantolu Madonna
['roman', 'tarih']
Sabahattin Ali
['Ayşe', 'Can', 'Ece']
TypeError: 'tuple' object does not support
item assignment
```

Yapı	Projedeki görevi
dict	ISBN → kitap: anında bul
set	Kategoriler: tekrar yok
tuple	Değişmeyen kayıt
list	Sıralı bekleme listesi

Projede kitaplar **dict**'te: 10.000 kitap olsa da ISBN ile arama tek adım.

Önce sadece veri ve fonksiyon

```
kitaplar = [
    {"isbn": "9789750719387",
     "ad": "Kürk Mantolu Madonna", "adet": 2},
    {"isbn": "9789754587494",
     "ad": "Sefiller", "adet": 1},
]
def odunc_ver(kitaplar, isbn):
    for k in kitaplar:
        if k["isbn"] == isbn and k["adet"] > 0:
            k["adet"] -= 1
            return True
    return False

print(odunc_ver(kitaplar, "9789754587494"))
print(odunc_ver(kitaplar, "9789754587494"))
kitaplar[0]["adet"] = -3     # kimse durdurmadı
print(kitaplar[0])
```

▷ ÇIKTI

```
True
False
{'isbn': '9789750719387',
 'ad': 'Kürk Mantolu Madonna',
 'adet': -3}
```

Çalışıyor, ama…

Veri (sözlük) ve davranış (fonksiyon) **ayrı**. Kurallar fonksiyonun içinde; veriye doğrudan dokunan herkes onları **atlar**.

HAFTA 2 · KARAR

Neden nesne yönelimli?

	Prosedürel	Nesne yönelimli
Veri ve davranış	Ayrı yerlerde	Aynı sınıfta
Kurallar	Fonksiyonda; atlanabilir	Nesnede; atlanamaz
Bir kitabı bulmak	Listeyi tara	kutup[isbn]
Yazdırmak	Sözlüğün ham hâli	Okunur metin (__str__)
Büyüme	Fonksiyonlar çoğalır, dağılır	Yeni sınıf, yeni metot

☆ Küçük betikler için prosedürel yeterli; **kuralları olan ve büyüyen** sistemler için nesneler.

BÖLÜM 2

02

Sınıf ve nesne

Hafta 3–4: __init__, self, değişkenler, metotlar

__init__ | self | sınıf değişkeni | @staticmethod | @classmethod

İlk Kitap sınıfı

```
class Kitap:
    def __init__(self, isbn, ad, yazar, adet=1):
        self.isbn = isbn
        self.ad = ad
        self.yazar = yazar
        self.adet = adet

k1 = Kitap("9789750719387", "Kürk Mantolu Madonna",
           "Sabahattin Ali", 2)
k2 = Kitap("9789754587494", "Sefiller", "Victor Hugo")
print(k1.ad, k1.adet)
print(k2.ad, k2.adet)
print(k1.__dict__)
```

▷ ÇIKTI

```
Kürk Mantolu Madonna 2
Sefiller 1
{'isbn': '9789750719387',
 'ad': 'Kürk Mantolu Madonna',
 'yazar': 'Sabahattin Ali', 'adet': 2}
```

self = “bu nesne”

k1 oluşurken self, k1’i; **k2** oluşurken k2’yi gösterir. Her nesne kendi **__dict__**’ine sahiptir.

adet=1 varsayılan değer: k2’ye adet vermedik, 1 oldu.

Ortak sayaç ve ilk davranış

```python
class Kitap:
    toplam = 0                          # sınıf değişkeni
    def __init__(self, isbn, ad, adet=1):
        self.isbn = isbn                # örnek değişkenleri
        self.ad = ad
        self.adet = adet
        Kitap.toplam += 1
    def odunc_ver(self):                # örnek metodu
        if self.adet == 0:
            return False
        self.adet -= 1
        return True

k1 = Kitap("9789750719387", "Kürk Mantolu Madonna", 2)
k2 = Kitap("9789754587494", "Sefiller")
print(Kitap.toplam, k1.toplam)
print(k2.odunc_ver(), k2.odunc_ver(), k2.adet)
```

▷ ÇIKTI

```
2 2
True False 0
```

Sınıf değişkeni

toplam tüm kitaplar için **tek**. Artırırken **Kitap.toplam** yazdık, self.toplam değil.

Örnek metodu

Veri ve davranış artık birlikte: stok kuralı kitabın **kendi metodunda**.

Statik ve sınıf metodu

```python
class Kitap:
    def __init__(self, isbn, ad, yazar, adet=1):
        if not Kitap.isbn_gecerli(isbn):
            raise ValueError("Geçersiz ISBN")
        self.isbn = isbn; self.ad = ad
        self.yazar = yazar; self.adet = adet
    @staticmethod
    def isbn_gecerli(isbn):
        return len(isbn) == 13 and isbn.isdigit()
    @classmethod
    def kayittan(cls, satir):
        isbn, ad, yazar, adet = satir.split(";")
        return cls(isbn, ad, yazar, int(adet))
print(Kitap.isbn_gecerli("9789750719387"),
      Kitap.isbn_gecerli("97897-5071"))
k = Kitap.kayittan("9789754587494;Sefiller;Victor Hugo;1")
print(k.ad, k.yazar, k.adet, type(k.adet).__name__)
```

▷ ÇIKTI

```
True False
Sefiller Victor Hugo 1 int
```

Tür	İlk parametre	Ne zaman?
Örnek	self	Nesnenin verisi
Sınıf	cls	Alternatif kurucu
Statik	yok	Yardımcı hesap

HAFTA 4 · TUZAK

Sınıf değişkeninde liste: herkes paylaşır

```
class Uye:
    odunc = []            # YANLIŞ: sınıfta tek liste
    def __init__(self, ad):
        self.ad = ad

a = Uye("Ayşe"); b = Uye("Can")
a.odunc.append("Sefiller")
print(b.odunc, a.odunc is b.odunc)
```

▷ **ÇIKTI** `['Sefiller'] True`

```
class Uye:
    def __init__(self, ad):
        self.ad = ad
        self.odunc = []   # DOĞRU: her üyeye ayrı
```

Kural

Herkes için **aynı** olan (LIMIT, toplam) → sınıf değişkeni. Her nesneye **özel** olan (ödünç listesi) → **__init__** içinde self.

BÖLÜM 3

03 Kapsülleme

Hafta 5: kuralları nesnenin içine kilitlemek

__isbn _adet @property .setter salt okunur tuple

ISBN değişmez, adet doğrulanır

```
class Kitap:
    def __init__(self, isbn, ad, yazar, adet=1):
        ...
        self.__isbn = isbn
        self.adet = adet        # setter'dan geçer
    @property
    def isbn(self):
        return self.__isbn
    @property
    def adet(self):
        return self._adet
    @adet.setter
    def adet(self, deger):
        if deger < 0:
            raise ValueError("Adet negatif olamaz")
        self._adet = deger
```

```
k = Kitap("9789751024008", "Nutuk",
          "Atatürk", 3)
print(k.isbn, k.adet)
k.adet = -5
k.isbn = "1111111111111"
print(list(k.__dict__))
```

▷ ÇIKTI

```
9789751024008 3
Hata: Adet negatif olamaz
Hata: property 'isbn' of 'Kitap'
object has no setter
['_Kitap__isbn', 'ad', 'yazar', '_adet']
```

Uye: limit ve korunan ödünç listesi

```python
class Uye:
    LIMIT = 2
    def __init__(self, ad):
        self.ad = ad
        self.__odunc = []
    @property
    def odunc(self):
        return tuple(self.__odunc)
    def al(self, kitap):
        if len(self.__odunc) >= Uye.LIMIT:
            raise ValueError(f"{self.ad}: limit dolu")
        self.__odunc.append(kitap)

ayse = Uye("Ayşe")
ayse.al("Nutuk"); ayse.al("Sefiller")
print(ayse.odunc)
ayse.al("Kürk Mantolu Madonna")
```

▷ ÇIKTI

```
('Nutuk', 'Sefiller')
Hata: Ayşe: limit dolu
```

Dışarıya kopya

odunc bir **tuple** döndürür: üye listesini okuyabilirsiniz ama **append** ile limiti atlayamazsınız.

Hafta 4 + 5

LIMIT sınıf değişkeni (herkese aynı kural), **__odunc** özel örnek değişkeni (her üyeye ayrı).

HAFTA 5 · TEST

Sistemi bozmaya çalışalım

Saldırı	Sonuç	Kim durdurdu?
Kitap("123", ...)	Hata: Geçersiz ISBN	isbn_gecerli()
k.adet = -5	Hata: Adet negatif olamaz	adet.setter
k.isbn = "111..."	Hata: ... has no setter	@property
ayse.al(...) × 3	Hata: Ayşe: limit dolu	Uye.LIMIT
ayse.odunc.append(...)	Hata: 'tuple' object has no attribute 'append'	tuple(...)
k.__isbn = ...	Yeni, ilgisiz nitelik açılır	isim karıştırma

☆ Prosedürel sürümdeki **adet = -3** açığı kapandı. Arka kapı **k._Kitap__isbn** hâlâ var: kazara değil, ancak **kasıtla** açılır.

BÖLÜM 4

04

Magic metotlar

Hafta 6: sınıfları Python'un diline uydurmak

`__str__` `__repr__` `__eq__ / __hash__` `__lt__` `__len__, __contains__` `__iadd__`

Kitap kendini tanıtıyor ve karşılaştırılıyor

```
    def __str__(self):
        return f"{self.ad} ({self.yazar}) · {self.adet} adet"
    def __repr__(self):
        return f"Kitap({self.isbn!r}, {self.ad!r})"
    def __eq__(self, o):
        if not isinstance(o, Kitap):
            return NotImplemented
        return self.isbn == o.isbn
    def __hash__(self):
        return hash(self.isbn)
    def __lt__(self, o):
        return self.ad < o.ad
a = Kitap("9789754587494", "Sefiller", "Victor Hugo")
b = Kitap("9789754587494", "Sefiller", "V. Hugo", 4)
print(a); print(repr(a))
print(a == b, a is b, len({a, b}))
print(sorted([a, nutuk]))
```

▷ ÇIKTI

```
Sefiller (Victor Hugo) · 1 adet
Kitap('9789754587494', 'Sefiller')
True False 1
[Kitap('9789751024008', 'Nutuk'),
 Kitap('9789754587494', 'Sefiller')]
```

Eşitlik = aynı ISBN

Yazar farklı yazılsa da ISBN aynıysa **aynı kitap**. __hash__ da ISBN'den: kümede tek eleman.

Kütüphane bir koleksiyon gibi davranıyor

```
class Kutuphane:
    def __init__(self, ad):
        self.ad = ad
        self._kitaplar = {}            # isbn -> Kitap
    def __iadd__(self, kitap):
        if kitap in self:
            self[kitap.isbn].adet += kitap.adet
        else:
            self._kitaplar[kitap.isbn] = kitap
        return self
    def __len__(self):
        return len(self._kitaplar)
    def __contains__(self, kitap):
        return kitap.isbn in self._kitaplar
    def __getitem__(self, isbn):
        return self._kitaplar[isbn]
    def __iter__(self):
        return iter(sorted(self._kitaplar.values()))
```

```
kutup = Kutuphane("Merkez")
kutup += nutuk                 # 3 adet
kutup += Kitap("9789754587494",
         "Sefiller", "Victor Hugo")
kutup += Kitap("9789751024008",
         "Nutuk", "Atatürk", 2)
print(len(kutup), nutuk in kutup)
print(kutup["9789751024008"])
print([k.ad for k in kutup])
```

▷ ÇIKTI

```
2 True
Nutuk (Atatürk) · 5 adet
['Nutuk', 'Sefiller']
```

Ödünç verme ve iade

```
    def odunc_ver(self, uye, isbn):
        kitap = self[isbn]
        if kitap.adet == 0:
            raise ValueError(f"{kitap.ad}: stokta yok")
        uye.al(kitap)          # limit doluysa hata
        kitap.adet -= 1        # en son değiştir
    def iade_al(self, uye, isbn):
        kitap = self[isbn]
        uye.ver(kitap)         # almamışsa hata
        kitap.adet += 1
```

Önce kontrol, sonra değişiklik

uye.al() limit hatası verirse **kitap.adet** hiç azalmaz. Sıra ters olsaydı stoktan kitap kaybolurdu.

Görev paylaşımı

Kutuphane **stoku**, Uye **limiti**, Kitap **adedin negatif olmamasını** denetler. Hiçbiri diğerinin verisine doğrudan dokunmaz.

self[isbn]: sınıf kendi **__getitem__**'ini kullanıyor.

05

BÖLÜM 5

Hepsi bir arada

Tam kod, senaryolar ve hafta haritası

Kitap · Uye · Kutuphane · senaryo · hata testleri

TAM KOD · 1/3

Kitap sınıfı

```
class Kitap:
    toplam = 0
    def __init__(self, isbn, ad, yazar, adet=1):
        if not Kitap.isbn_gecerli(isbn):
            raise ValueError("Geçersiz ISBN")
        self.__isbn = isbn
        self.ad = ad
        self.yazar = yazar
        self.adet = adet
        Kitap.toplam += 1
    @staticmethod
    def isbn_gecerli(isbn):
        return len(isbn) == 13 and isbn.isdigit()
    @classmethod
    def kayittan(cls, satir):
        isbn, ad, yazar, adet = satir.split(";")
        return cls(isbn, ad, yazar, int(adet))
    @property
    def isbn(self):
        return self.__isbn
```

```
    @property
    def adet(self):
        return self._adet
    @adet.setter
    def adet(self, deger):
        if deger < 0:
            raise ValueError("Adet negatif olamaz")
        self._adet = deger
    def __str__(self):
        return f"{self.ad} ({self.yazar}) · {self.adet} adet"
    def __repr__(self):
        return f"Kitap({self.isbn!r}, {self.ad!r})"
    def __eq__(self, o):
        if not isinstance(o, Kitap):
            return NotImplemented
        return self.isbn == o.isbn
    def __hash__(self):
        return hash(self.isbn)
    def __lt__(self, o):
        return self.ad < o.ad
```

TAM KOD · 2/3

Uye sınıfı

```python
class Uye:
    LIMIT = 2
    def __init__(self, ad):
        self.ad = ad
        self.__odunc = []
    @property
    def odunc(self):
        return tuple(self.__odunc)
    def al(self, kitap):
        if len(self.__odunc) >= Uye.LIMIT:
            raise ValueError(f"{self.ad}: limit dolu")
        self.__odunc.append(kitap)
    def ver(self, kitap):
        if kitap not in self.__odunc:
            raise ValueError(f"{self.ad} bu kitabı almamış")
        self.__odunc.remove(kitap)
    def __str__(self):
        return f"{self.ad} · {len(self.__odunc)} kitap"
```

Yeni: ver()

Üye almadığı kitabı iade edemez. Kontrolü listeden silmeden **önce** yapıyoruz.

Yeni: __str__

print(ayse) → Ayşe · 1 kitap

Uye hiçbir kitabın adedine dokunmaz; bu iş Kutuphane'nin.

TAM KOD · 3/3

Kutuphane sınıfı

```
class Kutuphane:
    def __init__(self, ad):
        self.ad = ad
        self._kitaplar = {}
    def __iadd__(self, kitap):
        if kitap in self:
            self[kitap.isbn].adet += kitap.adet
        else:
            self._kitaplar[kitap.isbn] = kitap
        return self
    def __len__(self):
        return len(self._kitaplar)
    def __contains__(self, kitap):
        return kitap.isbn in self._kitaplar
    def __getitem__(self, isbn):
        return self._kitaplar[isbn]
    def __iter__(self):
        return iter(sorted(self._kitaplar.values()))
```

```
    def odunc_ver(self, uye, isbn):
        kitap = self[isbn]
        if kitap.adet == 0:
            raise ValueError(f"{kitap.ad}: stokta yok")
        uye.al(kitap)
        kitap.adet -= 1
    def iade_al(self, uye, isbn):
        kitap = self[isbn]
        uye.ver(kitap)
        kitap.adet += 1
```

SENARYO · NORMAL GÜN

Sistem çalışıyor

```
kutup = Kutuphane("Merkez")
kutup += Kitap("9789750719387", "Kürk Mantolu Madonna",
               "Sabahattin Ali", 2)
kutup += Kitap.kayittan(
    "9789754587494;Sefiller;Victor Hugo;1")
kutup += Kitap("9789750719387", "Kürk Mantolu Madonna",
               "Sabahattin Ali")
kutup += Kitap("9789751024008", "Nutuk", "Atatürk", 3)
print(len(kutup), Kitap.toplam)
for k in kutup:
    print(k)
ayse = Uye("Ayşe")
kutup.odunc_ver(ayse, "9789754587494")
print(ayse, ayse.odunc)
print(kutup["9789754587494"])
```

▷ ÇIKTI

```
3 4
Kürk Mantolu Madonna (Sabahattin Ali)
  · 3 adet
Nutuk (Atatürk) · 3 adet
Sefiller (Victor Hugo) · 1 adet
Ayşe · 1 kitap
  (Kitap('9789754587494', 'Sefiller'),)
Sefiller (Victor Hugo) · 0 adet
```

3 4: kütüphanede 3 farklı kitap, ama 4 Kitap nesnesi oluşturuldu. Tekrar eden Kürk Mantolu Madonna adedi artırdı.

SENARYO · KÖTÜ GÜN

Hatalar doğru yerde yakalanıyor

```
can = Uye("Can")
kutup.odunc_ver(can, "9789754587494")       # Sefiller
Kitap("123", "Deneme", "Biri")
kutup["9789751024008"].adet = -5
kutup.odunc_ver(ayse, "9789751024008")      # Nutuk
kutup.odunc_ver(ayse, "9789750719387")      # 3. kitap
ayse.odunc.append("Hile")
kutup["0000000000000"]
print(kutup["9789750719387"].adet,
      kutup["9789751024008"].adet)
kutup.iade_al(ayse, "9789754587494")
print(ayse, kutup["9789754587494"].adet)
kutup.iade_al(can, "9789751024008")
```

▷ ÇIKTI

```
Hata: Sefiller: stokta yok
Hata: Geçersiz ISBN
Hata: Adet negatif olamaz
Hata: Ayşe: limit dolu
Hata: 'tuple' object has no attribute
'append'
Hata: KeyError '0000000000000'
3 2
Ayşe · 1 kitap 1
Hata: Can bu kitabı almamış
```

Limit hatasında Kürk Mantolu Madonna hâlâ **3**: önce kontrol, sonra değişiklik.

ADIM ADIM

Tek çağrı, altı hafta

```
kutup.odunc_ver(ayse, "9789751024008")
```

HAFTA 6 `self[isbn]` `__getitem__` → **HAFTA 1** dict'te anahtar arama → **HAFTA 5** `kitap.adet` getter → **HAFTA 4** `uye.al()` `Uye.LIMIT` → **HAFTA 5** `adet -= 1` setter

Hafta 3

Bütün bunlar **self** sayesinde doğru nesnede oluyor: kutup, ayse ve kitap kendi verilerini taşıyor.

Hafta 2

Kodu çağıran kişi ayrıntıları bilmiyor; sadece **ne istediğini** söylüyor. Bu, OOP'nin soyutlama gücü.

HAFTA HARİTASI

Projedeki her parça hangi haftadan?

Projede	Kavram	Hafta
self._kitaplar = {}	Sözlük, anahtarla arama	1
class Kitap / odunc_ver()	Prosedürelden OOP'ye	2
__init__(self, ...)	Yapıcı ve self	3
toplam, LIMIT	Sınıf değişkeni	4
isbn_gecerli, kayittan	Statik ve sınıf metodu	4
__isbn, @property	Kapsülleme, salt okunur	5
tuple(self.__odunc)	Sızmayan getter	5
__str__, __eq__, __lt__	Gösterim, karşılaştırma	6
__iadd__, __len__, __iter__	Kapsayıcı davranış	6

ÇIKTIYI TAHMİN ET · 1/2

Hafta 4 ve 5'ten

SORU 1 · HAFTA 4

```
class A:
    n = 0

a, b = A(), A()
a.n = 5
A.n = 10
print(a.n, b.n)
```

5 10: a kendi n'ini aldı; b sınıfınkini okuyor.

SORU 2 · HAFTA 5

```
class K:
    def __init__(self):
        self.__x = 1

k = K()
k.__x = 2
print(k.__x, k._K__x)
```

2 1: dışarıdaki __x karıştırılmaz.

ÇIKTIYI TAHMİN ET · 2/2

Hafta 5 ve 6'dan

SORU 3 · HAFTA 6

```
class P:
    def __init__(self, v):
        self.v = v
    def __add__(self, o):
        return P(self.v + o.v)
    def __repr__(self):
        return f"P({self.v})"

print(sum([P(1), P(2)], P(0)))
```

P(3): başlangıç P(0) verildi; __radd__ gerekmedi.

SORU 4 · HAFTA 5

```
class T:
    def __init__(self, c):
        self.c = c
    @property
    def c(self):
        return self._c
    @c.setter
    def c(self, v):
        self._c = max(-273, v)

print(T(-500).c, T(25).c)
```

-273 25: __init__ de setter'dan geçer.

SÖZLÜK

Altı haftanın temel terimleri

Terim	İngilizce	Bir cümleyle	Hafta
Sözlük	dictionary	Anahtar → değer eşlemesi	1
Paradigma	paradigm	Programı düşünme ve kurma biçimi	2
Yapıcı	constructor	Nesne oluşurken çalışan __init__	3
Örnek / sınıf değişkeni	instance / class var.	Nesneye özel / herkese ortak veri	4
Statik / sınıf metodu	static / class method	self almayan / cls alan metot	4
Kapsülleme	encapsulation	Veriyi kurallı arayüzle korumak	5
Özellik	property	Nitelik gibi kullanılan metot	5
Özel metot	magic / dunder method	Python'un kendiliğinden çağırdığı __ad__	6
Operatör aşırı yükleme	operator overloading	Operatöre sınıfa özel anlam vermek	6

DİKKAT

Altı haftanın en sık altı hatası

`def ad(): # self yok`

X.ad() takes 0 positional arguments but 1 was given (hafta 3)

`kitaplar=[] varsayılanı`

Bütün nesneler **aynı listeyi** paylaşır; None verip içeride oluşturun. (hafta 3)

`odunc = [] # sınıfta`

Bir üyenin aldığı kitap herkesinkinde görünür. (hafta 4)

`return self.adet`

Property kendini çağırır: **maximum recursion depth exceeded** (hafta 5)

`__str__ içinde print`

__str__ returned non-string (type NoneType) (hafta 6)

`__iadd__'de return yok`

kutup += kitap sonrası kutup **None** olur. (hafta 6)

KONTROL LİSTESİ

Ara sınavdan önce kendime sorduklarım

- list, tuple, set, dict arasında **seçim yapabiliyor** muyum?
- Prosedürel ve OOP yaklaşımını **karşılaştırabiliyor** muyum?
- **self**'in neyi gösterdiğini açıklayabiliyor muyum?
- Sınıf ve örnek değişkeninin **farkını** çıktıyla gösterebiliyor muyum?
- **@staticmethod** ile **@classmethod**'u ayırabiliyor muyum?
- **_ad**, **__ad** ve isim karıştırmayı biliyor muyum?
- Doğrulayan bir **property/setter** yazabiliyor muyum?
- **__str__**, **__eq__**, **__add__**, **__len__** yazabiliyor muyum?

ARA SINAVA HAZIRLIK

Nasıl çalışmalı?

1

Elle yaz

Kütüphane kodunu bakmadan, boş bir dosyaya baştan yazın.

2

Tahmin et

Her haftanın tahmin slaytını çalıştırmadan önce kâğıtta çözün.

3

Boz ve düzelt

Sık hatalar slaytlarındaki hataları bilerek yapın, mesajlarını okuyun.

4

Anlat

Bir arkadaşınıza tek bir satırın arkasında hangi haftanın çalıştığını anlatın.

Kendinizi bu soru tipleriyle deneyin

Çıktı tahmini (verilen kod ne yazar?), **hata bulma** (bu kod neden çalışmaz?), **kod tamamlama** (eksik metodu yazın) ve **sınıf tasarlama** (gereksinimden sınıf çıkarın).

ÖZET

Bu hafta neler yaptık?

Temeller

Doğru veri yapısı ve prosedürelden OOP'ye geçiş kararı.

Sınıf ve nesne

Kitap, Uye, Kutuphane; sınıf değişkenleri ve üç metot türü.

Kapsülleme

Salt okunur ISBN, doğrulanan adet, sızmayan ödünç listesi.

Magic metotlar

print, ==, sıralama, len, in, [ ], for ve += ile doğal kullanım.

İyi bir sınıf **kendi kurallarını bilir**, **kendini anlatır** ve **Python gibi davranır**.

ÖDEV

Projeyi genişletin

1 • Arama

Kutuphane.ara(kelime): adında veya yazarında kelime geçen kitapları sıralı liste olarak döndürsün.

2 • Toplu yükleme

@classmethod Kutuphane.metinden(ad, metin): her satırı Kitap.kayittan ile ekleyen alternatif kurucu.

3 • Kategori

Kitap'a kategoriler (set) ekleyin; dışarıya frozenset ya da tuple olarak verin.

4 • Rapor

Kutuphane.__str__: ad, kitap sayısı, toplam adet ve ödünçteki kitap sayısı.

GELECEK HAFTA · 8

Genel tekrar ve ara sınav

1–7. haftalar. Sorularınızı yazılı getirin.

Hazırlık: bu haftanın tam kodunu bakmadan yeniden yazın.